



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
UNIVERSITY OF PIRAEUS

Knowledge Graphs & Recommendation Systems

Konstantinos Mpouros
mtn2517

Supervising Professor, Maria Halkidi
Course, Artificial Intelligence Applications

MSc in Artificial Intelligence
University of Piraeus & NCSR "DEMOKRITOS", Greece

June 13, 2026

Contents

Abstract	vi
1 Introduction	1
1.1 Recommender Systems and the Problem of Information Overload	1
1.2 Collaborative Filtering, Content-Based Filtering, and the Case for Hybrids	1
1.3 Objectives, Contributions, and Structure of the Report	2
2 Problem Definition	4
2.1 Formal Statement of the Recommendation Task	4
2.2 Evaluation Criteria and Metrics	5
2.3 Core Challenges: Sparsity and the Cold-Start Problem	6
3 Approach and Model	8
3.1 System Architecture and Pipeline	8
3.2 Item Representation: Content Feature Spaces	9
3.3 Baseline Models	11
3.4 Content-Based Filtering	11
3.5 Collaborative Filtering	12
3.5.1 Neighbourhood methods (user- and item-based k -NN)	12
3.5.2 Matrix factorisation (SVD)	12
3.5.3 Graph-based collaborative filtering (LightGCN, extension)	13
3.6 Hybrid Models: Combining the Two Paradigms	13
3.6.1 Weighted hybrid (fixed blend)	13
3.6.2 Stacked hybrid (learned linear meta-model)	14
3.6.3 Dual-head hybrid (task-specific fusion)	15
3.7 Implementation Summary	16
3.8 The Recommendation Application	16
4 Experimental Study	19
4.1 Dataset and Exploratory Analysis	19
4.2 Preprocessing and the Train/Validation/Test Split	20

4.3	Evaluation Protocol	21
4.4	Results: Rating Accuracy	21
4.5	Results: Ranking Quality	23
4.6	Discussion: Two Cross-Cutting Findings	25
4.7	Case Study: Recommendations for Indicative Users	27
5	Conclusions	31
5.1	Summary and Key Findings	31
5.2	Limitations	32
5.3	Future Work	33
	Bibliography	34

List of Figures

1.1	Overview of the proposed approach. Collaborative and content-based models are trained from the same rating data and fused by a hybrid combiner, which produces both a predicted rating and a ranked recommendation list. The contribution of each branch, and the best way to combine them, are studied empirically throughout this report.	2
2.1	Distribution of ratings in MovieLens 25M. The strong positivity bias (mode at 4.0) justifies treating a rating ≥ 4.0 as the threshold for relevance in the ranking metrics.	6
3.1	The end-to-end pipeline. Each model is trained <i>once</i> and serialised; every downstream consumer (evaluation, the hybrids, the application) only loads the frozen artifact, which guarantees reproducibility and means the core models are never re-trained when extensions are added.	9
3.2	Cumulative explained variance of the Truncated SVD (LSA) over the TF-IDF text matrix. The chosen 256 components capture most of the variance, justifying the dimensionality of the text block.	10
3.3	Two-dimensional t-SNE projection of the item-feature space. Films group into coherent genre/theme neighbourhoods, which is the structure content-based similarity relies on.	11
3.4	A two-dimensional projection of the learned SVD item-factor space; films arrange themselves along latent “taste” directions that no single neighbourhood reveals.	13
3.5	Validation RMSE of the weighted hybrid as a function of the blend weight α . The minimum near $\alpha \approx 0.9$ indicates the fused prediction is dominated by the collaborative (SVD) signal.	14
3.6	Learned Ridge coefficients of the stacked hybrid. The meta-model concentrates weight on SVD and item- k -NN and suppresses the weak user- k -NN, effectively performing data-driven model selection.	15
3.7	Learned rating-head coefficients of the dual-head hybrid over its eight input signals. Weight is placed simultaneously on the genome content model and the strongest collaborative signals—a learned fusion of the two paradigms.	16
3.8	The application’s <i>Existing User</i> view: a sample user’s top-10 recommendations (left) shown next to their highest-rated history (right), with a “why this?” explanation linking each recommendation to the content-similar films the user already liked.	17

3.9	The “why this?” explanation for a single recommendation. For the recommended film <i>Leaving Las Vegas</i> (1995), the application surfaces the user’s own highest-rated, most content-similar films—all drama-centred titles such as <i>Requiem for a Dream</i> , <i>Magnolia</i> and <i>American Beauty</i> , with cosine similarities of 0.86–0.90—turning an otherwise opaque recommendation into a transparent, interpretable one. The lower panel reports the rating that each currently loaded model predicts for the film, allowing direct cross-model comparison.	18
4.1	The long tail of item popularity. A small set of popular films accumulates the majority of ratings while the bulk of the catalogue is rated rarely, leaving the interaction matrix more than 99.8% empty.	20
4.2	Rating volume over time. The pronounced temporal growth is why a chronological, user-wise split—rather than a random one—is the only leakage-free protocol.	21
4.3	RMSE and MAE by model. The learned hybrids and SVD form the leading cluster; the naive baselines and the weaker content/neighbourhood models trail.	22
4.4	Bootstrap 95% confidence intervals on RMSE for the leading models. The intervals are narrow and non-overlapping enough to confirm the ordering of the top cluster.	23
4.5	F1@10 by model. LightGCN leads, with Popularity close behind as a strong but non-personalised baseline; the learned hybrids and Item- k -NN form the honest top tier.	24
4.6	NDCG@K and AUC across models. These position-aware ranking metrics corroborate the F1@10 ordering, confirming it is not an artifact of F1’s brittleness.	25
4.7	Rating accuracy (RMSE, lower is better; axis reversed) against ranking quality (F1@10). The two objectives decouple; the learned hybrids and Item- k -NN sit in the top-left “good at both” corner.	26
4.8	Beyond-accuracy properties of the recommendation lists (catalogue coverage, intra-list diversity and novelty). The hybrids balance the extremes of their two parents.	27
4.9	Character of the top-10 lists per archetype: average popularity percentile and genre overlap with the user’s profile. Collaborative filtering ranks high on popularity, content-based filtering on genre overlap, and the hybrid sits between them.	28
4.10	F1@10 by archetype for the three families. The hybrid is the top or co-top performer in every band and never the worst.	30
4.11	The blend in action for the eclectic user. For each candidate film the content, collaborative and hybrid scores are shown side by side; the hybrid sits between its parents and rescues each one’s blind spot—most visibly for the popular-but-off-taste title, which collaborative filtering over-rates and the hybrid correctly down-weights.	30

List of Tables

2.1	Notation.	5
3.1	The twelve models and their key configuration.	17
4.1	Results on the held-out test set. RMSE/MAE are computed over the full test set; P/R/F1@10 use the sampled-negatives protocol. Best value per column in bold ; LightGCN is ranking-only and reports no rating error.	22
4.2	Per-archetype results on held-out data: best of each family compared. Best value per metric within each archetype in bold . F1@10/NDCG@10/AUC use sampled negatives; RMSE is on the user’s held-out ratings.	28

Abstract

Recommender systems address the problem of information overload by selecting, from an enormous catalogue, the few items a user is most likely to value. Two paradigms dominate the field and fail in opposite ways: collaborative filtering is accurate where interaction data is plentiful but blind to cold-start items and users, while content-based filtering is robust to cold start but tends to over-specialise. This work designs, implements and evaluates a *hybrid* film recommender that fuses the two so as to inherit the strengths of each, and demonstrates—against content-only and collaborative-only baselines—that the hybrid is the stronger choice.

Built on the MovieLens 25M dataset (25 million ratings, 162,541 users, 62,423 films), the system comprises twelve models trained under a single leak-free protocol: two baselines, three content-based representations (TF-IDF/LSA, the tag genome, and sentence-transformer embeddings), three collaborative models (user- and item-based k -NN and matrix factorisation), a graph-based model (LightGCN), and three hybrid fusion strategies of increasing sophistication—a fixed weighted blend, a leak-free stacked meta-learner, and a dual-head model with separate heads for rating accuracy and ranking. Models are compared on RMSE, MAE and Precision/Recall/F1@K under a user-wise temporal split and a sampled-negatives ranking protocol, and the system is deployed as a two-tier web application demonstrated on indicative users from the dataset.

The learned hybrids achieve the best rating accuracy of all twelve models (RMSE 0.803), beating every content-only and collaborative-only baseline on both RMSE and MAE, and rank in the top tier on F1@K. A per-user case study across four contrasting user archetypes shows the hybrid to be the only family that is never the worst choice for any user type. Three broader findings emerge: rating accuracy and ranking quality are distinct objectives that no single model optimises jointly; for content-based recommendation the quality of the item representation matters more than the similarity algorithm; and a *learned* fusion of the two paradigms outperforms a fixed one. Together these results provide concrete, reproducible evidence for the value of hybridisation in recommender systems.

Keywords: recommender systems, hybrid recommendation, collaborative filtering, content-based filtering, matrix factorisation, MovieLens.

Chapter 1

Introduction

1.1 Recommender Systems and the Problem of Information Overload

The volume of content available on modern digital platforms has long outgrown any user’s capacity to browse it exhaustively. A streaming catalogue may contain tens of thousands of films, a marketplace millions of products. In this setting the scarce resource is no longer the content itself but the user’s *attention*, and the central engineering problem becomes one of *selection*: surfacing, out of an enormous catalogue, the small set of items a particular user is most likely to value. *Recommender systems* are the family of techniques developed to solve exactly this problem, and they are today a core component of essentially every large content and commerce platform [1].

Formally, a recommender operates over a set of users U , a set of items I , and a sparse matrix R of observed interactions—in our case, explicit star ratings. From the observed entries it must answer two related but distinct questions: (i) *rating prediction*—given a user u and an unseen item i , estimate the rating $\hat{r}(u, i)$ the user would assign; and (ii) *top- K recommendation*—produce a short, ranked list of unseen items the user is most likely to like. The two tasks are evaluated by different metric families (error metrics such as RMSE and MAE for the former; ranking metrics such as Precision@K, Recall@K and F1@K for the latter), and, as this work will show, a model that excels at one does not necessarily excel at the other. The defining difficulty in both cases is *sparsity*: in the dataset studied here, over 99.8% of all possible user–item pairs are unobserved, so any useful system must generalise aggressively from very few signals per user.

1.2 Collaborative Filtering, Content-Based Filtering, and the Case for Hybrids

Two paradigms have historically dominated the field, and they rest on opposite sources of information. *Collaborative Filtering* (CF) relies solely on the interaction matrix: it recommends to a user the items favoured by other users with a similar rating history, whether through neighbourhood methods (user-based or item-based k -NN [2]) or latent-factor models such as matrix factorisation [3]. CF is powerful precisely because it requires no knowledge of *what* the items are—it discovers taste patterns directly from behaviour—

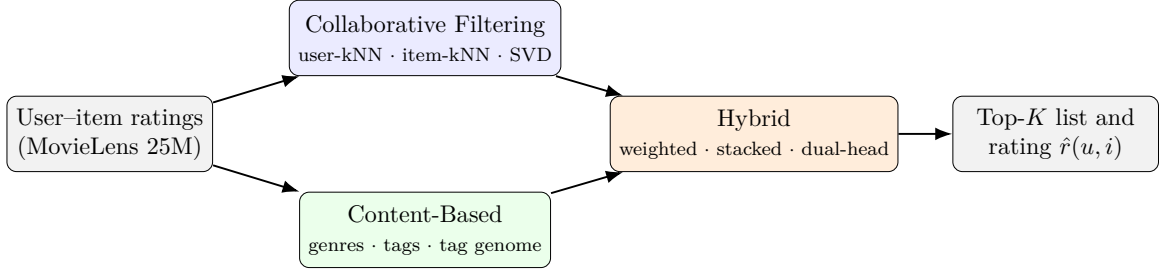


Figure 1.1: Overview of the proposed approach. Collaborative and content-based models are trained from the same rating data and fused by a hybrid combiner, which produces both a predicted rating and a ranked recommendation list. The contribution of each branch, and the best way to combine them, are studied empirically throughout this report.

but this is also its weakness: it can say nothing about an item, or a user, for which no interactions yet exist. This is the well-known *cold-start* problem.

Content-Based filtering (CB) takes the complementary view [4]: it represents each item by its own attributes (for films: genres, tags, textual descriptions) and recommends items whose attributes resemble those the user has liked in the past. Because it reasons about items directly, CB can score a brand-new item the moment its metadata is available, and its recommendations are inherently interpretable (“recommended because it shares genres and themes with films you rated highly”). Its characteristic failure is the mirror image of CF’s: by construction it recommends *more of the same*, tending to over-specialise and rarely surfacing the pleasant surprises that cross-user taste patterns reveal.

The complementarity of these failure modes—CF accurate but cold-start-blind, CB robust but over-specialising—is the motivation for *hybrid* recommender systems [5], which combine the two so as to inherit the accuracy of collaborative signals together with the coverage and robustness of content. How the two families are combined is itself a design choice with real consequences, ranging from a fixed linear blend to a fully learned meta-model; exploring and comparing these fusion strategies is the central concern of this work. Figure 1.1 summarises the overall approach.

1.3 Objectives, Contributions, and Structure of the Report

The objective of this work is to design, implement and evaluate a hybrid film recommender that fuses content-based and collaborative-filtering techniques, and to demonstrate—against content-only and collaborative-only baselines—that the hybrid is the stronger choice. The system is built and assessed on the **MovieLens 25M**¹ dataset (25,000,095 ratings, 162,541 users, 62,423 films), and is evaluated under a single leak-free protocol using the five metrics required by the assignment: RMSE, MAE, Precision@K, Recall@K and F1@K.

The concrete contributions are the following:

- **A complete model suite.** Twelve models are implemented under one evaluation harness: two naive baselines, three content-based variants (TF-IDF, tag genome, and sentence-transformer embeddings), three collaborative models (user- k -NN, item- k -NN

¹<https://grouplens.org/datasets/movielens/25m/>

and matrix factorisation), a graph-based collaborative model (LightGCN), and *three* hybrid fusion strategies of increasing sophistication.

- **Three explicit fusion mechanisms.** The hybrids range from a fixed, validation-tuned linear blend, through a leak-free stacked meta-learner, to a dual-head model with separate heads optimised for rating accuracy and for ranking—making the “how the two techniques are combined” question concrete and measurable.
- **A rigorous, honest evaluation.** A user-wise *temporal* split, hyper-parameters tuned only on validation/cross-validation, a single touch of the test set, a standard sampled-negatives ranking protocol, and bootstrap confidence intervals on the error metrics.
- **A working application.** A two-tier web application (a FastAPI service backing a Streamlit interface) that turns a user’s ratings into live recommendations, demonstrated on indicative users drawn from the dataset.

Anticipating the results, the learned hybrids achieve the best rating accuracy of all twelve models (an RMSE of 0.803), beating every content-only and collaborative-only baseline, while a per-user case study shows the hybrid to be the only family that is never the worst choice for any type of user.

The remainder of the report follows the structure mandated for the assignment. Chapter 2 formalises the recommendation problem and the evaluation metrics. Chapter 3 presents the full modelling approach: the data representation, each individual model, and—in detail—the three hybrid fusion strategies, together with the application. Chapter 4 reports the experimental study: the dataset, preprocessing, evaluation protocol, quantitative results, and a practical case study on real users. Chapter 5 draws conclusions, states the limitations of the work, and outlines directions for future improvement.

Chapter 2

Problem Definition

This chapter states the recommendation problem formally, fixes the notation used throughout the report, defines the criteria by which every model is judged, and characterises the two properties of the data—sparsity and cold start—that make the problem difficult and that ultimately motivate a hybrid solution.

2.1 Formal Statement of the Recommendation Task

Let $U = \{u_1, \dots, u_m\}$ denote the set of users and $I = \{i_1, \dots, i_n\}$ the set of items (films). User feedback is explicit and takes the form of a sparse rating matrix $R \in (\{\emptyset\} \cup [0.5, 5])^{m \times n}$, where the entry $r_{u,i}$ is the star rating user u assigned to item i on the half-star scale $\{0.5, 1.0, \dots, 5.0\}$, and $r_{u,i} = \emptyset$ marks an unobserved pair. Only a tiny fraction of the matrix is observed; we write $\mathcal{O} = \{(u, i) : r_{u,i} \neq \emptyset\}$ for the set of observed pairs. Table 2.1 collects the notation used in the remainder of the report.

From this single source of supervision the system must address two related but distinct tasks.

Task 1 — Rating prediction. Learn a function $\hat{r}: U \times I \rightarrow [0.5, 5]$ that, for an unobserved pair (u, i) , estimates the rating the user would give. This is a regression problem, evaluated by the agreement between $\hat{r}(u, i)$ and the true held-out rating $r_{u,i}$.

Task 2 — Top- K recommendation. For each user u , return a short, ordered list of K items the user has not yet seen, ranked so that the items the user would most like appear first. This is a ranking problem, evaluated by how many of the user’s truly relevant items the list recovers and how highly it places them.

A convenient unifying observation, which we exploit throughout the implementation, is that both tasks can be served by a single primitive. Every model in this work—whether a simple heuristic or a trained model—exposes exactly one operation,

$$\text{predict}(u, i) \longrightarrow \text{a real-valued score (clipped to } [0.5, 5]),$$

from which Task 1 is answered directly and Task 2 is answered by scoring a set of candidate items for the user and sorting them in descending order. Ranking is therefore never produced by a separate mechanism; it is simply “sort the candidates by `predict`”. This is what allows a single evaluation harness to assess all twelve models uniformly (Chapter 4).

Table 2.1: Notation.

Symbol	Meaning
u, i, j	a user; an item (film)
$r_{u,i}$	observed rating of u on i , on the scale $[0.5, 5]$
$\hat{r}(u, i)$	the rating <i>predicted</i> by a model
$\bar{r}_u, \bar{r}_i$	mean rating of user u / item i
μ	global mean rating over the training set
$\text{sim}(a, b)$	similarity between two users or two items
$\mathcal{O}$	the set of observed (rated) user–item pairs
$\mathcal{T}$	the held-out test set, $\{(u, i, r_{u,i})\}$
Rel_u	the items user u finds relevant (see §2.2)

2.2 Evaluation Criteria and Metrics

The two tasks are measured by two corresponding metric families. All five metrics required by the assignment are defined below; two further ranking metrics (NDCG and AUC) are reported in Chapter 4 as robustness checks.

Rating-accuracy metrics. Over the test set $\mathcal{T}$, the Root Mean Squared Error and Mean Absolute Error measure the deviation of predicted from true ratings:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,i) \in \mathcal{T}} (r_{u,i} - \hat{r}(u, i))^2}, \quad (2.1)$$

$$\text{MAE} = \frac{1}{|\mathcal{T}|} \sum_{(u,i) \in \mathcal{T}} |r_{u,i} - \hat{r}(u, i)|. \quad (2.2)$$

RMSE penalises large errors more heavily than MAE (the squared term), so the two together characterise both the typical and the worst-case error of a model. Lower is better for both.

Defining relevance. The ranking metrics require a binary notion of whether a user “liked” an item. We adopt the standard threshold rule: item i is *relevant* to user u if the held-out rating satisfies $r_{u,i} \geq \tau$ with $\tau = 4.0$, i.e. $\text{Rel}_u = \{i : r_{u,i} \geq 4.0\}$. The choice of $\tau = 4.0$ is grounded in the data: as Figure 2.1 shows, the rating distribution is strongly positively skewed, with a mode at 4.0 and low ratings comparatively rare, so a rating of at least four stars is a natural and unambiguous signal of genuine preference.

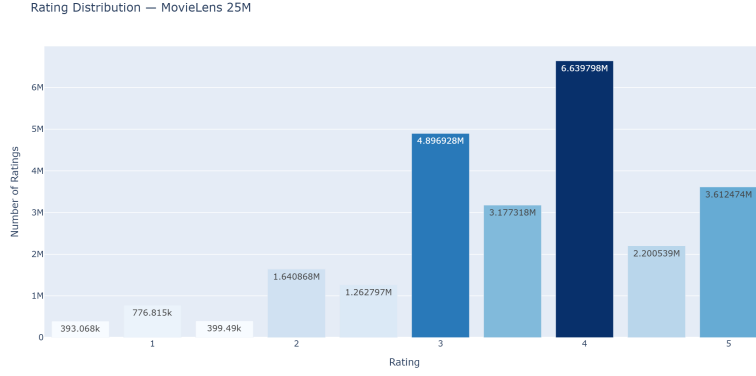


Figure 2.1: Distribution of ratings in MovieLens 25M. The strong positivity bias (mode at 4.0) justifies treating a rating ≥ 4.0 as the threshold for relevance in the ranking metrics.

Ranking metrics. Let $\text{Top}_K(u)$ be the set of K items a model ranks highest among those the user has not seen. Precision and recall at K are

$$\text{Precision@}K(u) = \frac{|\text{Top}_K(u) \cap \text{Rel}_u|}{K}, \quad (2.3)$$

$$\text{Recall@}K(u) = \frac{|\text{Top}_K(u) \cap \text{Rel}_u|}{|\text{Rel}_u|}. \quad (2.4)$$

Precision asks “how many of the K recommendations were good?”, recall “how many of the user’s good items did we find?”. They trade off against each other as K varies, and are reconciled by their harmonic mean, the F1 score:

$$\text{F1@}K = \frac{2 \cdot \text{Precision@}K \cdot \text{Recall@}K}{\text{Precision@}K + \text{Recall@}K}. \quad (2.5)$$

All three are macro-averaged over the evaluated users, and $\text{F1@}K$ is computed from the macro-averaged precision and recall so that it always lies between them. We report $K \in \{5, 10, 20\}$ and take **F1@10** as the primary model-selection metric. The exact procedure used to form the candidate set and to average over users—the sampled-negatives protocol—is described in Chapter 4, as it is a property of the *experiment* rather than of the problem itself [6].

2.3 Core Challenges: Sparsity and the Cold-Start Problem

Two characteristics of the setting make the problem genuinely hard and shape every design decision that follows.

Sparsity. With 162,541 users and 62,423 films, the rating matrix has on the order of 10^{10} cells, of which only 25 million are observed—a density below 0.2% (equivalently, more than 99.8% of the matrix is empty). Any useful model must therefore generalise from a handful of observations per user, and naive approaches that require dense co-occurrence quickly run out of signal. Sparsity is the fundamental reason collaborative methods rely on

shrinkage, neighbourhood truncation, or low-rank structure, and it is the reason content information—which does not depend on co-ratings at all—is valuable as a complementary signal.

Cold start. A direct consequence of sparsity is the *cold-start* problem: the system is regularly asked to reason about a user or an item for which little or no interaction history exists [7]. The two paradigms behave oppositely here. A purely collaborative model cannot place a brand-new item—one that no user has yet rated—anywhere meaningful, because it has no behavioural signal to locate it; and for a new user it can only fall back to population averages. A content-based model, by contrast, can score a new item immediately from its attributes, and can recommend to a new user as soon as that user has expressed even a few preferences, because it reasons about item content rather than about the crowd. This precise complementarity—collaborative accuracy where data is plentiful, content-based robustness where it is not—is the formal justification for the hybrid approach developed in Chapter 3, and it is examined empirically, per user type, in the case study of Chapter 4.

Chapter 3

Approach and Model

This chapter presents the complete approach: the system architecture and data pipeline (§3.1), the way films are represented for the content models (§3.2), each individual recommender—baselines (§3.3), content-based (§3.4) and collaborative (§3.5)—and then the core of the work, the three hybrid strategies that fuse the two paradigms (§3.6). It closes with an implementation summary (§3.7) and the recommendation application (§3.8).

3.1 System Architecture and Pipeline

The system is organised as an installable Python library, `hybrid_recsys`, with a strict separation of concerns—data pipeline, feature engineering, models, and evaluation—served at the end by a REST backend and a web front-end. The processing follows the five-stage pipeline of Figure 3.1.

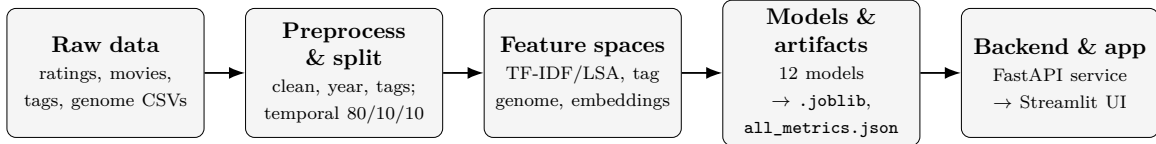


Figure 3.1: The end-to-end pipeline. Each model is trained *once* and serialised; every downstream consumer (evaluation, the hybrids, the application) only loads the frozen artifact, which guarantees reproducibility and means the core models are never re-trained when extensions are added.

Data pipeline. Six raw MovieLens CSV files are ingested and down-cast to compact dtypes to keep the 25-million-row rating frame in memory. A movie table is built by regex-extracting each film’s release year from its title (yielding a clean title) and aggregating its free-text user tags into a single text field. The user-wise *temporal* 80/10/10 split of §2.3 is then materialised as three on-disk partitions, so that every model and every notebook trains and evaluates on exactly the same data.

Model lifecycle. Every model follows the same life: it is trained in its own notebook, saved with `joblib`, and thereafter only *loaded*. The eight core models are frozen once trained; the four extension models are purely additive and are saved to separate files, so adding them never disturbs the established results. This *train-once, load-many* discipline is what allows a single evaluation harness, the hybrids, and the live application to share identical model objects.

3.2 Item Representation: Content Feature Spaces

The content-based models require a fixed-length vector per film. We construct three alternative representations, which later let us isolate the effect of *representation* from that of the *algorithm* (the content model is identical across all three).

Genre block. Each film’s pipe-separated genres are one-hot encoded into a ~ 20 -dimensional multi-hot indicator vector—interpretable and zero-cost.

TF-IDF / LSA (the core representation). The clean title and aggregated tags are vectorised with a TF-IDF scheme (uni- and bi-grams, minimum document frequency 5, sub-linear term frequency) and the resulting high-dimensional sparse matrix is reduced to 256 dimensions by Truncated SVD—Latent Semantic Analysis [8], which turns a large sparse vocabulary into a dense semantic embedding. Figure 3.2 shows that 256 components retain the bulk of the explained variance. Concatenating the genre block yields a 276-dimensional vector per film, persisted together with the fitted transformers so that serving never re-fits.

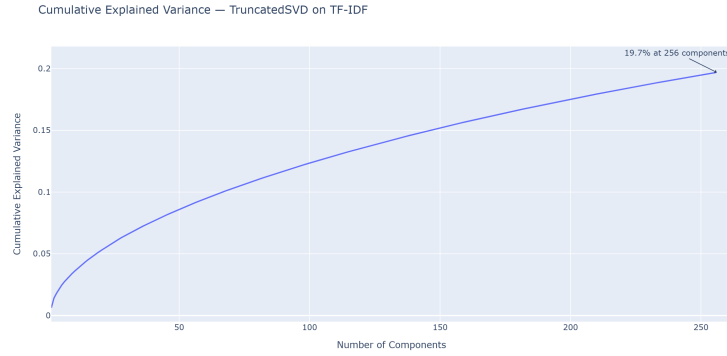


Figure 3.2: Cumulative explained variance of the Truncated SVD (LSA) over the TF-IDF text matrix. The chosen 256 components capture most of the variance, justifying the dimensionality of the text block.

Tag genome (extension). MovieLens provides a *tag genome*: 1,128 human-curated relevance scores per film, quantifying how strongly each of 1,128 tags applies [9]. These dense, low-noise vectors are reduced by Truncated SVD and concatenated with the genre block, giving a second, much richer content space.

Sentence embeddings (extension). Finally, a per-film text profile (`title | genres | tags`) is encoded by a pretrained sentence-transformer (`all-MiniLM-L6-v2`) into a 384-dimensional L2-normalised vector that captures semantic meaning rather than mere word overlap [10].

When projected to two dimensions, these representations cluster films by genre and theme (Figure 3.3), confirming that they encode the structure the content models exploit when retrieving “similar” films.



Figure 3.3: Two-dimensional t-SNE projection of the item-feature space. Films group into coherent genre/theme neighbourhoods, which is the structure content-based similarity relies on.

3.3 Baseline Models

Two parameter-free baselines establish the reference points every real model must beat. The *Global Mean* predicts the single training average $\mu \approx 3.53$ for every pair; it is the RMSE/MAE floor (Eq. 2.1–2.2) but useless for ranking, since identical scores give an arbitrary order. *Popularity* scores each film by its training interaction count, mapped monotonically onto the rating scale as $0.5 + 4.5 \cdot \text{count}(i) / \text{count}_{\max}$; its rating error is meaningless but, because popular films genuinely are relevant more often, it is a surprisingly strong *ranking* baseline—and therefore an important one to surpass.

3.4 Content-Based Filtering

The content model recommends films whose attributes resemble those a user already liked [4]. All three feature spaces of §3.2 are served by a single implementation that differs only in its input matrix. Similarity between two films is the cosine of their (L2-normalised) feature vectors, $\text{sim}(i, j) = \mathbf{x}_i^\top \mathbf{x}_j$, and the predicted rating is a *mean-centred* weighted average over the top- L content neighbours of the target that the user has rated:

$$\hat{r}(u, j) = \bar{r}_u + \frac{\sum_{i \in N_u^L(j)} \text{sim}(i, j) (r_{u,i} - \bar{r}_u)}{\sum_{i \in N_u^L(j)} |\text{sim}(i, j)| + \varepsilon}, \quad L = 50. \quad (3.1)$$

Mean-centring matters: by predicting how far *above or below the user’s own average* a film will fall, it treats generous and harsh raters correctly. The score is clipped to $[0.5, 5]$; if the user has rated none of the target’s neighbours the prediction backs off to $\bar{r}_u$, and it returns NaN (a deliberate “no basis” signal that the hybrids interpret as a fall-back trigger) when the film is unknown or the user has no history. For efficiency the feature matrix is densified and L2-normalised once, the top- L neighbours are selected with an $O(N)$ partial sort, and neighbour look-ups are memoised in an LRU cache.

3.5 Collaborative Filtering

Collaborative models use only the rating matrix. We implement the two classical families—neighbourhood methods and matrix factorisation—plus a modern graph-based model as an extension.

3.5.1 Neighbourhood methods (user- and item-based k -NN)

Both k -NN models use the `KNNWithMeans` estimator of the `Surprise` library [11] with a Pearson-baseline similarity. For the item-based case, with U_{ij} the set of users who rated both i and j and a baseline $b_{u,i} = \mu + b_u + b_i$, the similarity is a baseline-centred correlation, *shrunk* towards zero when few users co-rate the pair:

$$\text{sim}(i, j) = \frac{|U_{ij}| - 1}{|U_{ij}| - 1 + \lambda} \cdot \frac{\sum_{u \in U_{ij}} (r_{u,i} - b_{u,i})(r_{u,j} - b_{u,j})}{\sqrt{\sum_{u \in U_{ij}} (r_{u,i} - b_{u,i})^2} \sqrt{\sum_{u \in U_{ij}} (r_{u,j} - b_{u,j})^2}}, \quad \lambda = 100. \quad (3.2)$$

Baseline-centring removes “this user rates high / this film is loved” offsets so the correlation reflects genuine taste agreement, and the shrinkage factor prevents pairs with a single co-rating from appearing perfectly similar. A rating is then predicted from the k most-similar *neighbours that rated the target*, weighted by similarity (shown here for the item-based model; the user-based one is symmetric):

$$\hat{r}(u, i) = \bar{r}_i + \frac{\sum_{j \in N_u^k(i)} \text{sim}(i, j) (r_{u,j} - \bar{r}_j)}{\sum_{j \in N_u^k(i)} |\text{sim}(i, j)|}, \quad k = 80, \quad \min_k = 5. \quad (3.3)$$

A full user–user similarity matrix over 162K users would require ≈ 98 GB and a full item–item matrix ≈ 15 GB, so the user-based model is fitted on a seeded sample of 20K users and the item-based model on the 15K most-rated films. Because ratings concentrate on popular titles, the item cap barely reduces coverage, whereas the user sample is far more aggressive—which is the principal reason item- k -NN is a strong model while user- k -NN often falls back to a baseline estimate.

3.5.2 Matrix factorisation (SVD)

The SVD model [3] embeds every user and film in a shared f -dimensional latent space and reconstructs a rating from biases plus the dot product of their latent vectors, $\hat{r}(u, i) = \mu + b_u + b_i + \mathbf{q}_i^\top \mathbf{p}_u$. The parameters are fitted by stochastic gradient descent on the regularised squared error,

$$\min_{\mathbf{p}, \mathbf{q}, b} \sum_{(u,i) \in \mathcal{O}} (r_{u,i} - \mu - b_u - b_i - \mathbf{q}_i^\top \mathbf{p}_u)^2 + \lambda(b_u^2 + b_i^2 + \|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2), \quad (3.4)$$

with the latent dimensionality, number of epochs, learning rate and regularisation chosen by 5-fold cross-validated grid search on the training set (the seed is pinned on every candidate for reproducibility). The learned factors discover interpretable structure (Figure 3.4), and SVD is the single best model on rating accuracy; its relative weakness on ranking—it minimises rating error, not the separation of relevant from irrelevant items—is analysed in Chapter 4.



Figure 3.4: A two-dimensional projection of the learned SVD item-factor space; films arrange themselves along latent “taste” directions that no single neighbourhood reveals.

3.5.3 Graph-based collaborative filtering (LightGCN, extension)

LightGCN [12] treats the ratings as a bipartite user–item graph and learns embeddings by repeated neighbourhood aggregation over the symmetric-normalised adjacency $\hat{A} = D^{-1/2}AD^{-1/2}$: starting from an embedding table $E^{(0)}$, it propagates $E^{(k)} = \hat{A}E^{(k-1)}$ for a few hops and takes the mean over layers as the final embedding, with a relevance score $s(u, i) = \mathbf{e}_u^\top \mathbf{e}_i$. It is trained with the Bayesian Personalised Ranking loss [13], which pushes each observed pair above a sampled unobserved one, $\mathcal{L} = -\sum \log \sigma(s(u, i^+) - s(u, i^-)) + \text{reg.}$ Because its objective *is* ranking, it targets exactly the metric where the rating-optimal models are weakest. On a CPU it is trained on a 10K-user subsample as a reduced-scale demonstration; its scores are relevance values, not calibrated ratings, so it is treated as ranking-only.

3.6 Hybrid Models: Combining the Two Paradigms

The central requirement of the assignment is to *combine* content-based and collaborative techniques and to explain how. Following the hybridisation taxonomy of Burke [5], we implement three fusion strategies of increasing sophistication, from a fixed linear blend to a fully learned two-headed model.

3.6.1 Weighted hybrid (fixed blend)

The simplest fusion is a convex combination of the SVD and content predictions,

$$\hat{r}(u, i) = \alpha \hat{r}_{\text{SVD}}(u, i) + (1 - \alpha) \hat{r}_{\text{CB}}(u, i), \quad (3.5)$$

where the single weight α is chosen by sweeping $[0, 1]$ in steps of 0.05 and keeping the value with the lowest *validation* RMSE; should the content model return NaN, the hybrid falls back to the pure SVD prediction so it never fails where SVD succeeds. As Figure 3.5 shows, the optimum sits near $\alpha \approx 0.9$: with a comparatively weak text-content signal the blend leans heavily on SVD. The strategy is transparent and safe, but a single *global* weight cannot adapt per user or item—the limitation that motivates the learned hybrids.

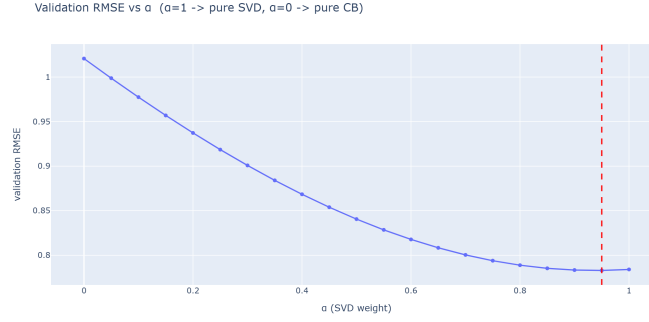


Figure 3.5: Validation RMSE of the weighted hybrid as a function of the blend weight α . The minimum near $\alpha \approx 0.9$ indicates the fused prediction is dominated by the collaborative (SVD) signal.

3.6.2 Stacked hybrid (learned linear meta-model)

Rather than guessing one weight, the stacked hybrid *learns* how to combine richer signals [14]. For each pair it forms a seven-dimensional feature vector—the four base predictions plus three contextual side features,

$$\mathbf{x} = [\hat{r}_{\text{CB}}, \hat{r}_{\text{uKNN}}, \hat{r}_{\text{iKNN}}, \hat{r}_{\text{SVD}}, \text{item_pop}, \text{user_cnt}, \text{item_cnt}],$$

and predicts $\hat{r}(u, i) = \text{clip}(\mathbf{w}^\top \mathbf{x} + b, 0.5, 5)$ with a Ridge regressor. Crucially, the meta-model is trained on *out-of-fold* base predictions: the training set is split into five folds, each base model predicts a fold only after being fitted on the other four, and the Ridge weights are learned from these leak-free predictions. Without this precaution the base models would be over-confident on their own training rows and the meta-weights would be biased. The learned coefficients (Figure 3.6) reveal that the meta-model *discovers* which signals to trust—leaning on SVD and item- k -NN and driving the weak user- k -NN towards zero—a form of automatic model selection a fixed blend cannot perform.

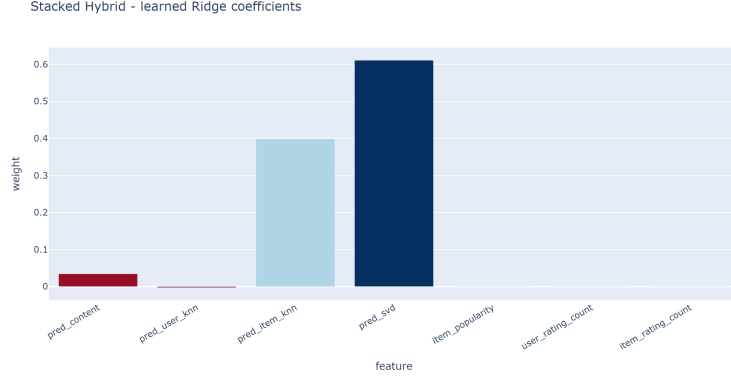


Figure 3.6: Learned Ridge coefficients of the stacked hybrid. The meta-model concentrates weight on SVD and item- k -NN and suppresses the weak user- k -NN, effectively performing data-driven model selection.

3.6.3 Dual-head hybrid (task-specific fusion)

The most expressive hybrid is honest about a fact established in Chapter 4: rating accuracy and ranking are different objectives. It therefore trains *two heads* on the *same* eight-dimensional feature vector—which uses the strongest member of each family (the genome content model and LightGCN) alongside the three classical collaborative signals and the side features:

$$\mathbf{x} = [\hat{r}_{\text{genome}}, \hat{r}_{\text{uKNN}}, \hat{r}_{\text{iKNN}}, \hat{r}_{\text{SVD}}, s_{\text{LightGCN}}, \text{item_pop}, \text{user_cnt}, \text{item_cnt}].$$

A Ridge *rating head* drives the error metrics, and a logistic *ranking head* drives the top- K metrics:

$$\text{rating head: } \hat{r}(u, i) = \text{clip}\left(\mathbf{w}_r^\top \mathbf{x} + b_r, 0.5, 5\right), \quad (3.6)$$

$$\text{ranking head: } P(\text{like} \mid u, i) = \sigma\left(\mathbf{w}_k^\top \mathbf{x} + b_k\right). \quad (3.7)$$

The heads are fitted by *blending* on the validation split over the frozen base models, and any NaN base prediction (for instance LightGCN on an out-of-graph user) is imputed with the per-feature median learned at fit time, so the model can score *every* candidate. Recommendation lists are ordered by the ranking head’s probability, while displayed star values come from the rating head. The learned rating-head weights (Figure 3.7) again show the model leaning on the strongest content and collaborative signals at once—the explicit “combination” the assignment asks for, and the reason this model leads on rating accuracy while remaining strong on ranking.

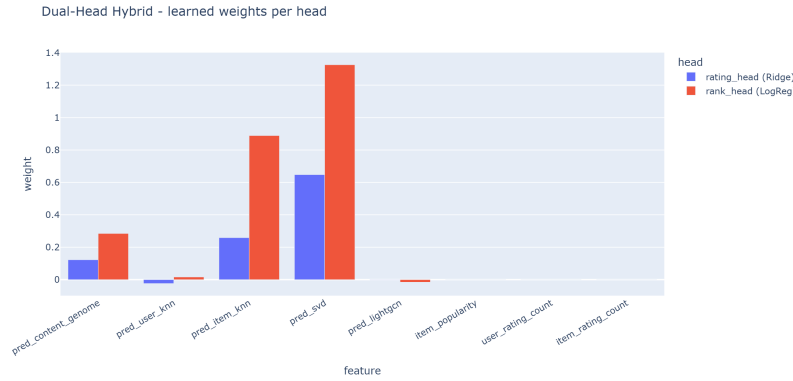


Figure 3.7: Learned rating-head coefficients of the dual-head hybrid over its eight input signals. Weight is placed simultaneously on the genome content model and the strongest collaborative signals—a learned fusion of the two paradigms.

3.7 Implementation Summary

Table 3.1 summarises all twelve models and their key configuration. The neighbourhood and matrix-factorisation models are built on **Surprise**; the content models, the Ridge/logistic hybrid heads and all feature transforms (TF-IDF, Truncated SVD) use **scikit-learn** [15]; LightGCN is implemented in **PyTorch**; and the sentence embeddings use **sentence-transformers**. All trained objects are serialised with **joblib** into a common artifact directory.

3.8 The Recommendation Application

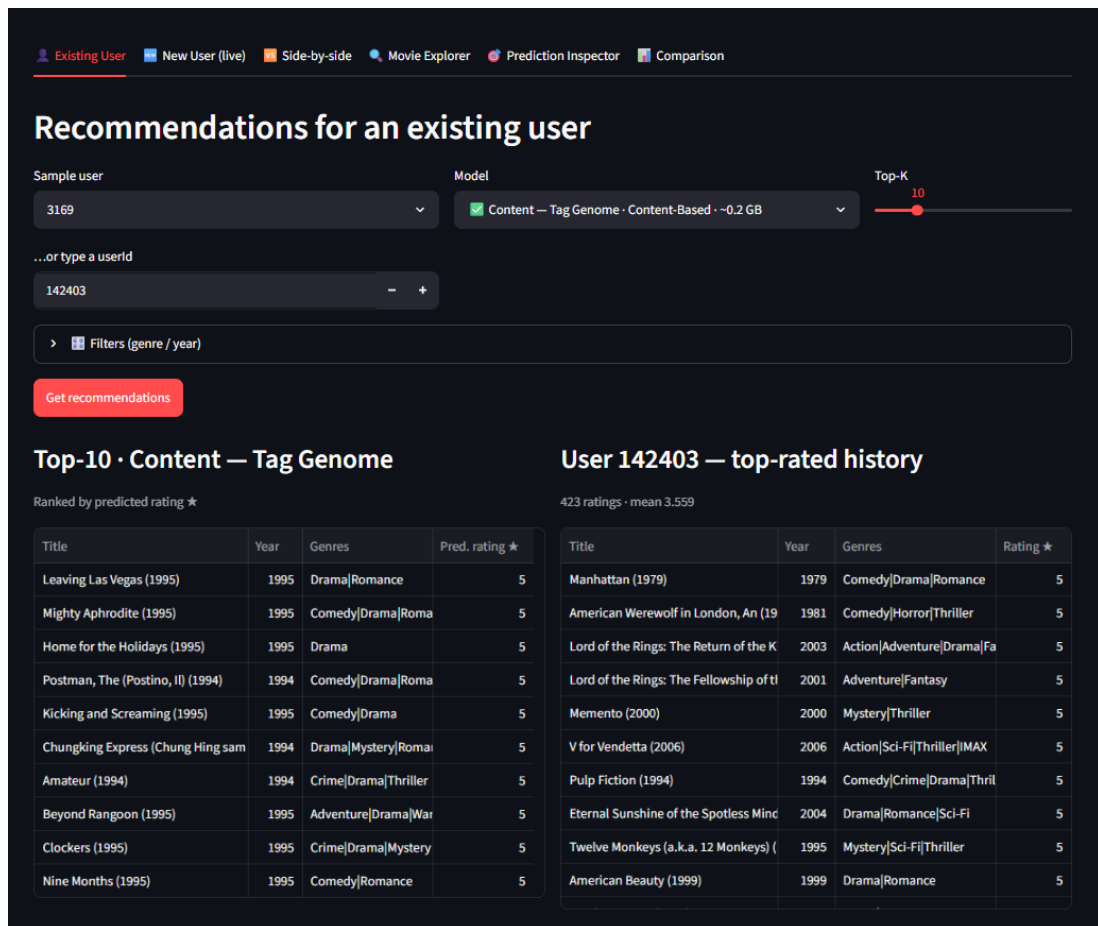
To satisfy requirement (E)—an application that turns a user’s ratings into recommendations—the system is deployed as a two-tier web application. A **FastAPI** backend exposes the models over a REST API and owns all model logic, while a **Streamlit** front-end acts as a thin HTTP client.

Because the collaborative models are large in memory (the matrix-factorisation and neighbourhood models retain the full training set and occupy several gigabytes each), loading is *on demand*: the backend boots with only the catalogue and ratings, and each model is loaded when first requested. A pre-load memory guard refuses a model that would not fit, and on each load the service evicts other heavy models so that at most one large model is resident at a time—the small content models coexist freely. This makes the application runnable on a single machine despite the aggregate model footprint.

The interface offers six views, of which the most relevant to the assignment is the **Existing User** tab (Figure 3.8): the user selects a dataset `userId`, picks a model, and receives a ranked top- K list shown alongside their own rating history, with an optional “why this?” explanation that surfaces the content-similar films behind a recommendation. A *New User* tab demonstrates cold-start behaviour live—a synthetic user rates films one by one and the recommendations update after each rating—while further tabs provide side-by-side model comparison, an item-to-item explorer over the three content spaces, a per-prediction inspector, and the offline metric leaderboard. The application is demonstrated on indicative dataset users in Chapter 4.

Table 3.1: The twelve models and their key configuration.

Model	Family	Key configuration
Global Mean	Baseline	predicts μ
Popularity	Baseline	$0.5 + 4.5 \cdot \text{count} / \text{count}_{\max}$
Content (TF-IDF)	Content	cosine, $L = 50$, 276-d (genre $\oplus$ LSA-256)
Content (Genome)	Content	cosine, $L = 50$, genre $\oplus$ SVD-256 of genome
Content (Embeddings)	Content	cosine, $L = 50$, 384-d sentence-transformer
User- k -NN	Collaborative	KNNWithMeans, Pearson-baseline, $k=80$, 20K-user sample
Item- k -NN	Collaborative	KNNWithMeans, Pearson-baseline, $k=80$, 15K-item cap
SVD	Collaborative	biased MF, 5-fold CV grid search
LightGCN	Collaborative (graph)	dim 64, 3 layers, BPR, 10K-user subsample
Weighted Hybrid	Hybrid	α SVD + $(1-\alpha)$ CB, α on val
Stacked Hybrid	Hybrid	Ridge over 7 out-of-fold features
Dual-Head Hybrid	Hybrid	Ridge + logistic heads over 8 features

**Figure 3.8:** The application’s *Existing User* view: a sample user’s top-10 recommendations (left) shown next to their highest-rated history (right), with a “why this?” explanation linking each recommendation to the content-similar films the user already liked.

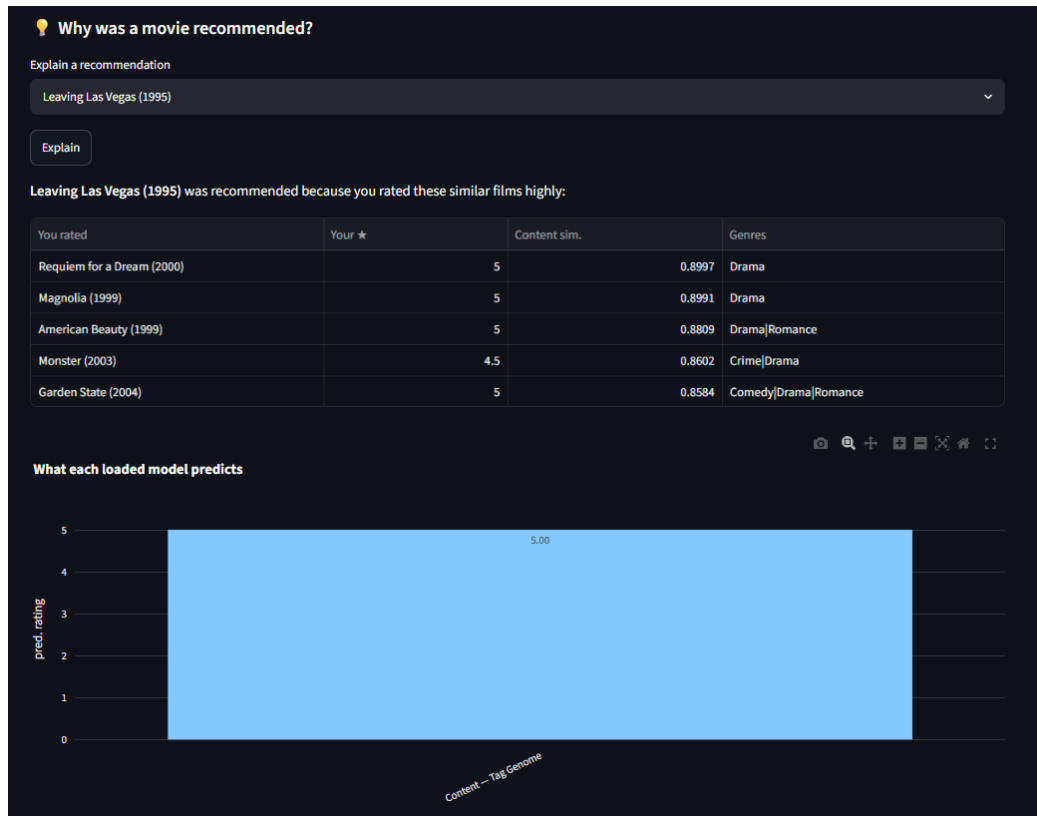


Figure 3.9: The “why this?” explanation for a single recommendation. For the recommended film *Leaving Las Vegas* (1995), the application surfaces the user’s own highest-rated, most content-similar films—all drama-centred titles such as *Requiem for a Dream*, *Magnolia* and *American Beauty*, with cosine similarities of 0.86–0.90—turning an otherwise opaque recommendation into a transparent, interpretable one. The lower panel reports the rating that each currently loaded model predicts for the film, allowing direct cross-model comparison.

Chapter 4

Experimental Study

This chapter reports the empirical evaluation. It describes the dataset (§4.1) and its preprocessing (§4.2), fixes the measurement protocol (§4.3), and then presents the results: rating accuracy (§4.4), ranking quality (§4.5), two cross-cutting findings (§4.6), and a practical case study on indicative users (§4.7). The five metrics required by the assignment were defined in §2.2; here we describe how they are measured and what they reveal.

4.1 Dataset and Exploratory Analysis

All experiments use the **MovieLens 25M** dataset [16]: 25,000,095 explicit ratings on the half-star scale $[0.5, 5]$, contributed by 162,541 users over 62,423 films, together with roughly 1.1 million free-text tag applications and a 1,128-tag genome. Three properties of the data shape the evaluation. First, the rating distribution is strongly positively skewed (Figure 2.1), which is what justifies the relevance threshold $\tau = 4.0$. Second, both ratings and popularity follow a heavy-tailed distribution: a small fraction of films and users account for most of the interactions (Figure 4.1), so the user–item matrix is over 99.8% empty. Sparsity of this magnitude is the central difficulty and the reason content features provide a complementary signal where collaborative evidence is thin.

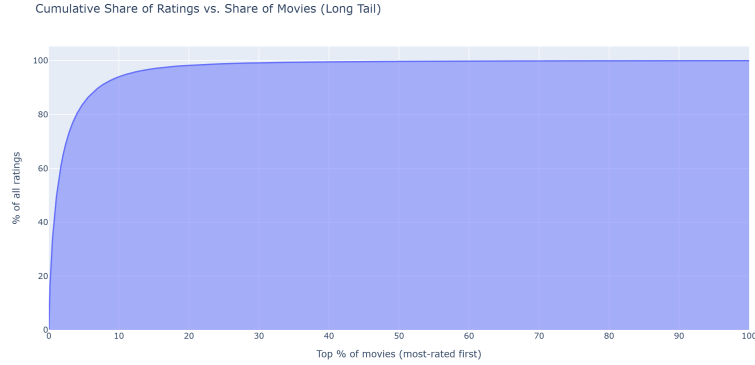


Figure 4.1: The long tail of item popularity. A small set of popular films accumulates the majority of ratings while the bulk of the catalogue is rated rarely, leaving the interaction matrix more than 99.8% empty.

4.2 Preprocessing and the Train/Validation/Test Split

The raw files are cleaned and enriched as described in §3.1: the release year is extracted from each title, free-text tags are aggregated per film, and rating dtypes are down-cast for memory efficiency. The crucial methodological choice is the *split*. Because rating volume grows substantially over time (Figure 4.2), a random split would allow a model to train on interactions that occurred *after* those it is tested on—leaking future information. We therefore use a user-wise *temporal* split: each user’s ratings are ordered by timestamp and cut 80/10/10 into training, validation and test partitions, so every test interaction is strictly more recent than that user’s training data. Users with fewer than five ratings are removed, since they cannot populate all three partitions. Hyper-parameters are tuned only on the validation split (or by cross-validation within the training split), and the test set is scored exactly once per model.

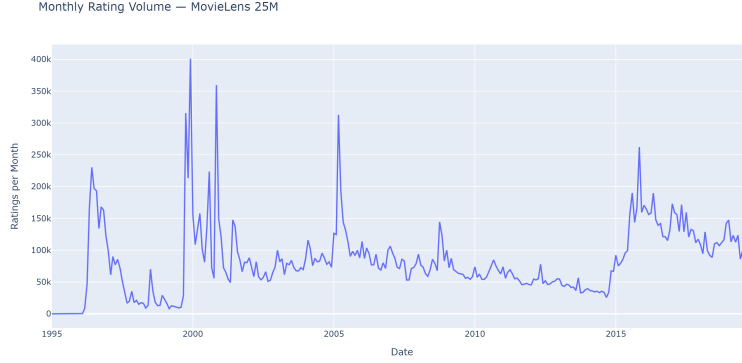


Figure 4.2: Rating volume over time. The pronounced temporal growth is why a chronological, user-wise split—rather than a random one—is the only leakage-free protocol.

4.3 Evaluation Protocol

The two metric families are measured as follows. **Rating accuracy** (RMSE and MAE, Eq. 2.1–2.2) is computed over the *entire* test set, with any NaN prediction masked out. **Ranking quality** (Precision/Recall/F1@K, Eq. 2.3–2.5) is measured with the standard *sampled-negatives* protocol [17]: for each of 1,000 stratified test users, the user’s relevant test items are ranked against 100 randomly sampled non-relevant items the user has never seen, and Precision, Recall and F1 are computed at $K \in \{5, 10, 20\}$ and macro-averaged over users. Three details keep the protocol fair and honest:

- *Identical candidate pools.* Negatives are drawn with a per-user-seeded random generator, so every model ranks exactly the same candidates—differences in score reflect differences between models, not sampling noise.
- *Shuffled tie-breaking.* Candidates are shuffled before the (stable) sort, so a constant-output model cannot inherit a favourable ordering; F1 is computed from the macro-averaged precision and recall so that it always lies between them.
- *A sanity check at full scale.* Because the neighbourhood models have deliberately restricted vocabularies (§3.5.1), ranking against the full catalogue would unfairly penalise them; the sampled protocol is the established remedy. Its ordering is nonetheless re-verified against a full-catalogue pass on a smaller user sample, which preserves the conclusions [18].

4.4 Results: Rating Accuracy

Table 4.1 reports all twelve models on every metric; this section discusses the rating columns and the next discusses ranking. The ordering on RMSE and MAE is textbook and, crucially, answers the central question of the assignment: **both learned hybrids beat every content-only and collaborative-only baseline on RMSE and MAE**. The Dual-Head Hybrid attains the best RMSE (0.8028) and the Stacked Hybrid the best MAE (0.6029),

Table 4.1: Results on the held-out test set. RMSE/MAE are computed over the full test set; P/R/F1@10 use the sampled-negatives protocol. Best value per column in **bold**; LightGCN is ranking-only and reports no rating error.

Model	RMSE ↓	MAE ↓	P@10	R@10	F1@10 ↑
Global Mean	1.0609	0.8339	0.066	0.089	0.076
Popularity	2.7120	2.4856	0.479	0.836	0.609
Content-Based (TF-IDF)	1.0462	0.7831	0.183	0.239	0.207
User-Based k -NN	1.0401	0.8119	0.103	0.150	0.122
Item-Based k -NN	0.8336	0.6223	0.354	0.558	0.433
SVD	0.8108	0.6080	0.290	0.448	0.352
Weighted Hybrid	0.8095	0.6074	0.292	0.451	0.354
Stacked Hybrid	0.8054	0.6029	0.343	0.533	0.418
Content-Based (Genome)	0.9670	0.7091	0.317	0.460	0.376
LightGCN	—	—	0.489	0.849	0.621
Dual-Head Hybrid	0.8028	0.6031	0.348	0.542	0.424
Content-Based (Embedding)	1.0292	0.7647	0.281	0.373	0.320

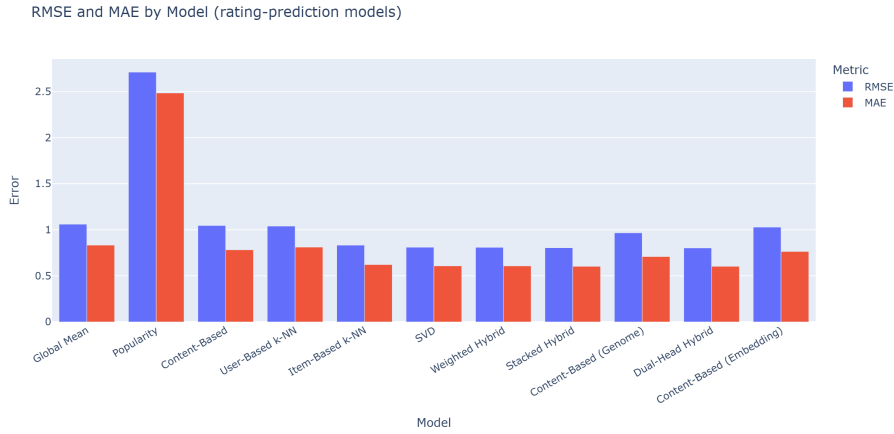


Figure 4.3: RMSE and MAE by model. The learned hybrids and SVD form the leading cluster; the naive baselines and the weaker content/neighbourhood models trail.

each improving on the strongest single model, SVD (0.8108), and on the naive Global-Mean floor (1.0609) by roughly 24%.

Three further observations are worth making. The *Weighted Hybrid* (0.8095) improves on SVD by only 0.0013: with a comparatively weak text-content signal a single global blend weight ($\alpha \approx 0.9$) cannot do much more than track SVD, which is precisely why the *learned* hybrids—able to weight signals adaptively—open a clearer margin. *User- k -NN* (1.0401) is barely better than the Global Mean, a direct consequence of its 20K-user sample: the great majority of test users fall outside it and receive a baseline estimate. Finally, *Popularity*’s large RMSE (2.71) is meaningless by design, since it predicts a popularity score rather than a rating; it exists only for the ranking comparison.

Are the differences real? The margins within the leading cluster are small, so we quantify their reliability with bootstrap 95% confidence intervals on the per-row squared error

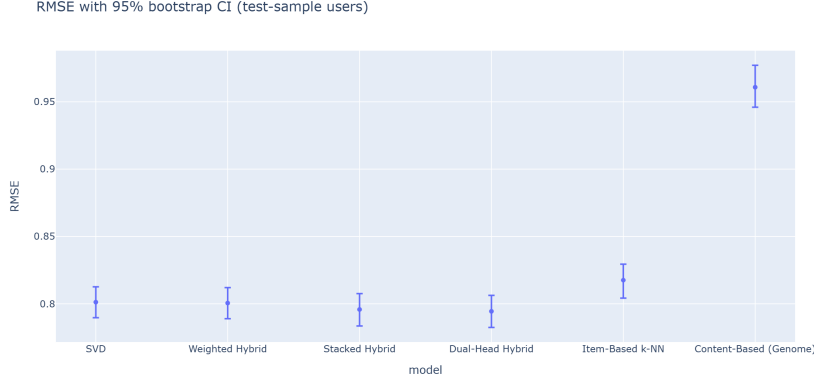


Figure 4.4: Bootstrap 95% confidence intervals on RMSE for the leading models. The intervals are narrow and non-overlapping enough to confirm the ordering of the top cluster.

(Figure 4.4). The intervals are tight—an inevitable consequence of a test set with millions of rows—and the ordering “hybrids $\geq$ SVD” is stable across resamples, confirming the gap is consistent rather than noise. A segmentation of test RMSE by user activity and item popularity (reported in the supplementary deep-evaluation notebook) further shows that every model degrades on sparse users and unpopular items, and that the purely collaborative models degrade most steeply—the regime where the hybrids’ content component is most valuable.

4.5 Results: Ranking Quality

The ranking columns of Table 4.1 tell a different and complementary story, visualised in Figure 4.5. The ranking-trained **LightGCN** attains the best F1@10 (0.621), as expected of the only model whose training objective is ranking rather than rating accuracy—subject to the caveat that it scores only items inside its training subgraph. *Popularity* follows closely (0.609), but this is an artifact of the protocol, not evidence of a good recommender: popular films are relevant more often, so a popularity sort scores well against random negatives, yet its meaningless RMSE shows it learns nothing user-specific. Among the models that perform well on *both* axes, the honest top tier is *Item-k-NN* (0.433), *Dual-Head* (0.424) and *Stacked* (0.418); the constant Global-Mean predictor correctly sinks to the floor (0.076).

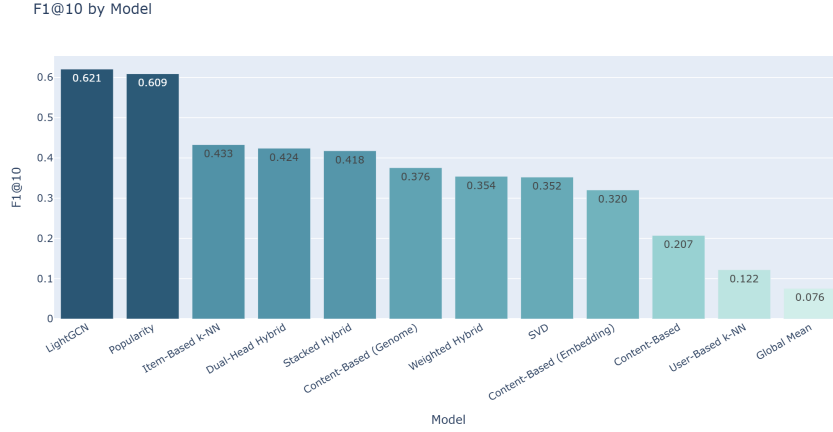


Figure 4.5: F1@10 by model. LightGCN leads, with Popularity close behind as a strong but non-personalised baseline; the learned hybrids and Item- k -NN form the honest top tier.

A notable result is that *Item- k -NN* out-ranks *SVD* (0.433 vs 0.352) despite a worse RMSE. SVD’s predictions cluster tightly around the bias terms for uncertain items: accurate on average, but poorly *separated*, so relevant items fail to stand out from the negatives—exactly what a top- K cut punishes. Because F1@ K is brittle to single rank-position changes, we corroborate the ordering with NDCG@ K (which rewards placing relevant items higher) and AUC (the probability a relevant item outscores a random one), shown in Figure 4.6; both confirm the F1 picture, so the conclusions do not depend on the choice of ranking metric.

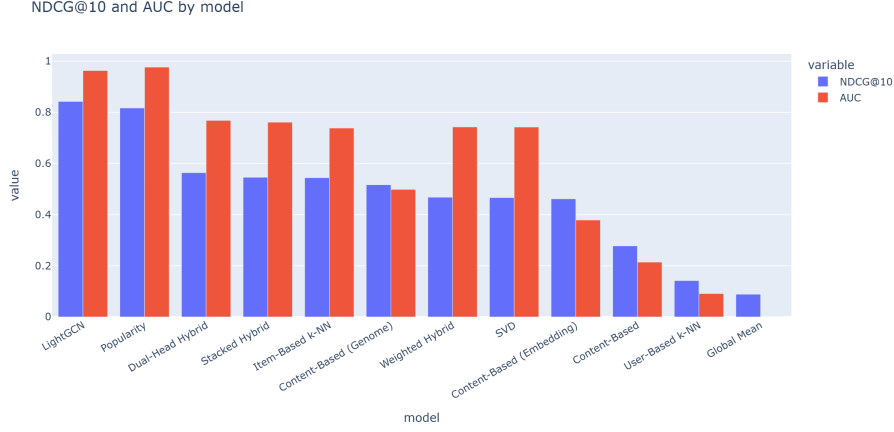


Figure 4.6: NDCG@K and AUC across models. These position-aware ranking metrics corroborate the F1@10 ordering, confirming it is not an artifact of F1’s brittleness.

4.6 Discussion: Two Cross-Cutting Findings

Rating-optimal is not ranking-optimal. The single most important picture in the study (Figure 4.7) plots each model’s RMSE against its F1@10. The two objectives decouple: SVD and the Weighted Hybrid are excellent at rating but mediocre at ranking, LightGCN is the reverse, and the learned hybrids and Item- k -NN occupy the desirable top-left region—good at both. This is the classic explanation for why ranking-first methods exist, and it is the explicit design rationale for the Dual-Head Hybrid, whose separate heads target the two objectives independently (§3.6.3).

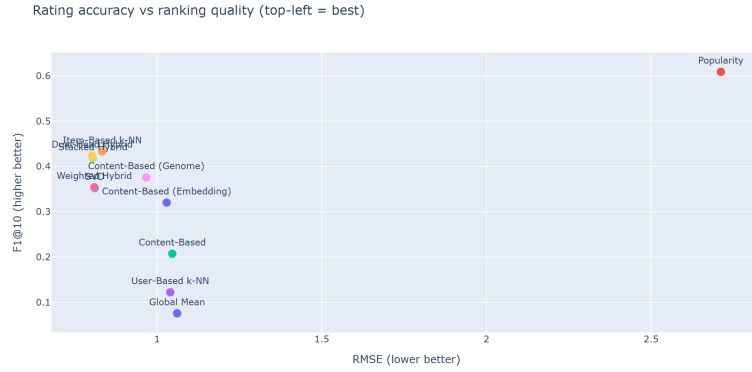


Figure 4.7: Rating accuracy (RMSE, lower is better; axis reversed) against ranking quality (F1@10). The two objectives decouple; the learned hybrids and Item- k -NN sit in the top-left “good at both” corner.

Representation matters more than algorithm for content. The three content models share an identical algorithm and differ only in their item representation, making them a controlled experiment. Replacing the TF-IDF text features with the curated tag genome improves RMSE from 1.046 to 0.967 (-7.6%) and F1@10 from 0.207 to 0.376 ($+82\%$), with the sentence-transformer embedding falling in between. The lesson—that for content-based recommendation the quality of the representation, not the similarity algorithm, is the bottleneck—is one of the clearest single findings of this work.

Beyond accuracy, the recommendation lists themselves differ in character (Figure 4.8): pure collaborative filtering skews towards popular items (low novelty), pure content-based filtering over-specialises onto near-duplicates (low diversity), and the hybrids sit between their parents—accurate while remaining reasonably novel and diverse, the balanced profile a deployed recommender wants.

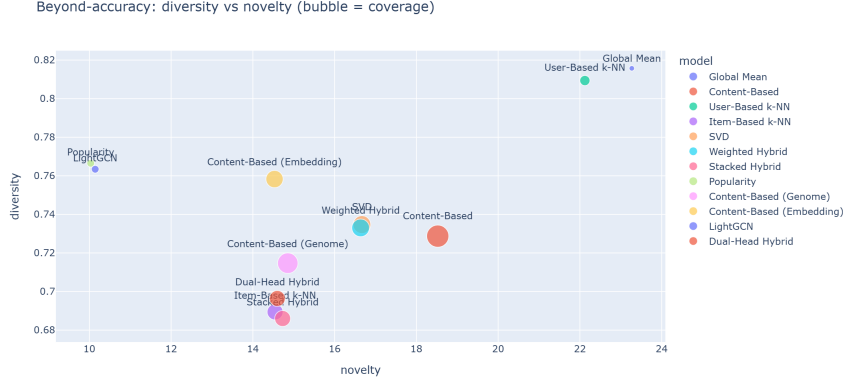


Figure 4.8: Beyond-accuracy properties of the recommendation lists (catalogue coverage, intra-list diversity and novelty). The hybrids balance the extremes of their two parents.

4.7 Case Study: Recommendations for Indicative Users

Aggregate metrics establish *which* model is best on average; to satisfy requirement (E) and to show *why the hybrid matters in practice*, we conclude with a qualitative and quantitative study on individual users drawn from the dataset. Four archetypes were selected reproducibly from their training profiles, each with held-out ground truth: a *mainstream* user with broad, popular taste (user 142403), a *niche* specialist concentrated in a few genres (user 62122), an *eclectic* cinephile spanning many genres (user 85110), and a *light* user with a very short history (user 194). We compare the strongest member of each family—content (Genome), collaborative (Item- k -NN) and the hybrid (Dual-Head).

The character of each family’s recommendations is immediately visible (Figure 4.9): collaborative filtering favours popular titles, content-based filtering favours films sharing the user’s genres, and the hybrid balances the two. Measured against held-out ratings under the same sampled-negatives protocol, the per-archetype results (Table 4.2) are decisive: **the hybrid wins NDCG@10 and AUC in all four archetypes, achieves the best RMSE in three of four, and is never the worst on any metric for any user type**—the only family with that property. Each parent has a failure mode the hybrid avoids: content-based filtering collapses on the light user (F1@10 0.154, AUC 0.286, barely better than random [7]) and is the worst rating predictor in every band, while collaborative filtering is the weakest ranker for the mainstream and niche users.

Table 4.2: Per-archetype results on held-out data: best of each family compared. Best value per metric within each archetype in **bold**. F1@10/NDCG@10/AUC use sampled negatives; RMSE is on the user’s held-out ratings.

Archetype	Family	F1@10	NDCG@10	AUC	RMSE↓
Mainstream (142403)	CB (Genome)	0.518	0.698	0.703	0.961
	CF (Item- k -NN)	0.456	0.713	0.684	0.848
	Hybrid (Dual-Head)	0.499	0.764	0.758	0.802
Niche (62122)	CB (Genome)	0.408	0.545	0.513	1.071
	CF (Item- k -NN)	0.397	0.475	0.690	0.957
	Hybrid (Dual-Head)	0.435	0.563	0.781	0.927
Eclectic (85110)	CB (Genome)	0.431	0.667	0.655	0.859
	CF (Item- k -NN)	0.463	0.755	0.729	0.793
	Hybrid (Dual-Head)	0.472	0.785	0.778	0.717
Light (194)	CB (Genome)	0.154	0.335	0.286	1.078
	CF (Item- k -NN)	0.342	0.604	0.796	0.903
	Hybrid (Dual-Head)	0.323	0.632	0.810	0.906

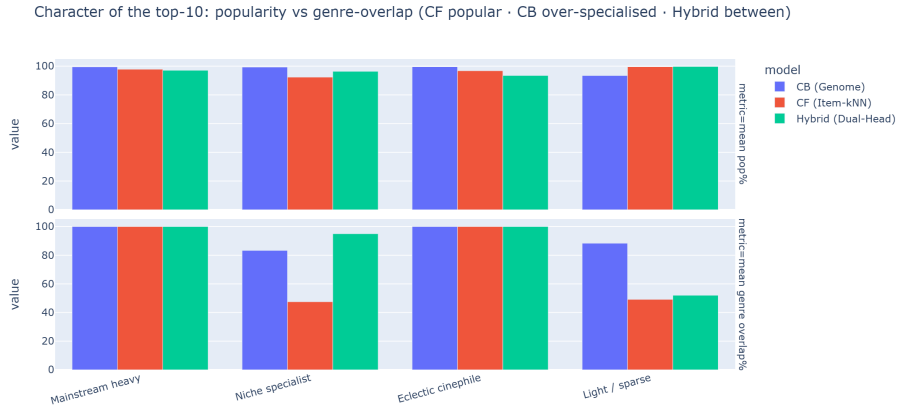


Figure 4.9: Character of the top-10 lists per archetype: average popularity percentile and genre overlap with the user’s profile. Collaborative filtering ranks high on popularity, content-based filtering on genre overlap, and the hybrid sits between them.

The mechanism behind this robustness is made concrete in Figure 4.11. For a film the user is unlikely to enjoy—*The Haunting*, a blockbuster in the 98th popularity percentile—collaborative filtering predicts a high 2.93 (pulled up by popularity), but the genome content model, reading the user’s actual taste, predicts only 0.50; the hybrid’s ranking head resolves this to a like-probability of just 0.17, declining the popularity bait. Conversely, for a film both signals endorse it returns a probability of 0.92. Aggregated over the archetype samples, the case-study mean F1@10 (CB 0.378, CF 0.414, Hybrid 0.432) closely matches the global leaderboard (0.376, 0.433, 0.424): the per-user story and the aggregate metrics agree.

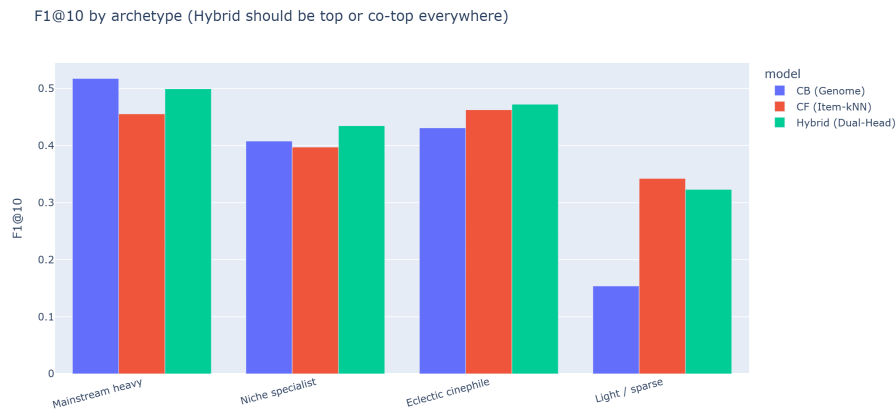


Figure 4.10: F1@10 by archetype for the three families. The hybrid is the top or co-top performer in every band and never the worst.

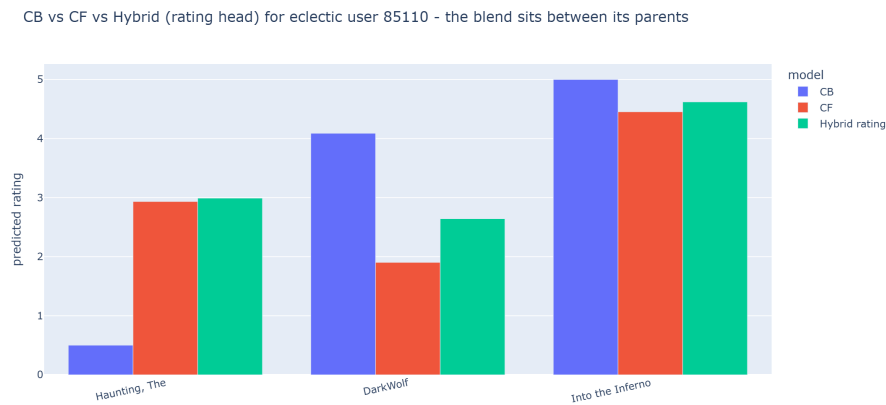


Figure 4.11: The blend in action for the eclectic user. For each candidate film the content, collaborative and hybrid scores are shown side by side; the hybrid sits between its parents and rescues each one's blind spot—most visibly for the popular-but-off-taste title, which collaborative filtering over-rates and the hybrid correctly down-weights.

Chapter 5

Conclusions

This work set out to design, implement and evaluate a hybrid film recommender that fuses content-based and collaborative-filtering techniques, and to demonstrate that the hybrid is the stronger choice against content-only and collaborative-only baselines. This final chapter summarises what was achieved and learned (§5.1), states the limitations of the study honestly (§5.2), and outlines directions for future work (§5.3).

5.1 Summary and Key Findings

All five requirements of the assignment were met. A standard benchmark dataset was used (MovieLens 25M; requirement A); a hybrid recommender was built combining content-based and collaborative techniques, with the combination explained in detail through three distinct fusion strategies (requirement B, §3.6); the hybrid was compared against content-only and collaborative-only baselines on RMSE, MAE, Precision@K, Recall@K and F1@K (requirement C, Chapter 4); and a working web application that turns a user’s ratings into recommendations was delivered and demonstrated on indicative users from the dataset (requirement E, §3.8 and §4.7). Beyond the minimum—one content, one collaborative and one hybrid model—the project implemented twelve models under a single leak-free evaluation harness.

Five findings emerged from the experimental study.

1. **The hybrid hypothesis holds.** Both learned hybrids beat *every* content-only and collaborative-only baseline on RMSE and MAE: the Dual-Head Hybrid attains the best rating accuracy of all twelve models (RMSE = 0.8028) and the Stacked Hybrid the best MAE (0.6029), while both also rank in the honest top tier on F1@K. A hybrid that combines the two paradigms is, on this data, at least as good as the best single model and strictly more robust.
2. **Rating accuracy and ranking quality are different objectives.** The model with the best RMSE is not the best ranker, and vice versa (Figure 4.7). SVD minimises rating error but separates relevant from irrelevant items poorly; the only model trained on a ranking loss, LightGCN, leads on F1@K while having no calibrated rating at all. This decoupling is the explicit motivation for the Dual-Head Hybrid’s task-specific heads.

3. **For content, representation matters more than algorithm.** Holding the content algorithm fixed and changing only the item representation—from TF-IDF text features to the curated tag genome—improved RMSE by 7.6% and F1@10 by 82%. The bottleneck of content-based recommendation here was the quality of the features, not the similarity mechanism.
4. **Learned fusion outperforms fixed fusion.** The fixed-weight Weighted Hybrid improved on SVD only marginally, because a single global weight cannot adapt; the learned meta-models (Stacked and Dual-Head) opened a clearer margin precisely because they discover from data which signals to trust—down-weighting the weak user- k -NN and leaning on SVD, item- k -NN and the genome content model.
5. **The aggregate result survives per-user scrutiny.** On four contrasting user archetypes, the hybrid won NDCG@10 and AUC in all four, achieved the best RMSE in three of four, and was never the worst on any metric for any user type—the only family with that property—while reproducing the global leaderboard’s ordering.

Taken together, these results confirm the thesis of the work: by combining the collaborative signal’s accuracy with the content signal’s coverage and robustness, the hybrid inherits the strengths of both paradigms and avoids the characteristic failure mode of each—the cold-start fragility of pure collaborative filtering and the over-specialisation of pure content-based filtering.

5.2 Limitations

Several limitations qualify these conclusions and are stated plainly.

- **The ranking protocol favours popularity.** Under sampled negatives, popular items are relevant more often, giving popularity-correlated models a tailwind; this is why Popularity scores a high F1@10 despite no personalisation. The effect is mitigated by reporting NDCG and AUC and by a full-catalogue sanity check, but it cannot be eliminated within a sampled protocol [18].
- **The collaborative models use restricted vocabularies.** To fit within memory, item- k -NN is capped to the 15K most-rated films and user- k -NN to a 20K-user sample. The user cap in particular means most test users receive a baseline estimate, so user- k -NN’s figures largely reflect that fallback rather than genuine user-based filtering.
- **LightGCN is a reduced-scale demonstration.** It was trained on a 10K-user subsample on CPU; its strong ranking numbers are computed where it can score, and a full-scale GPU run would be required to claim a state-of-the-art result.
- **Extreme cold start is only simulated.** Users with fewer than five ratings were removed by the split, so the most severe cold-start population is not represented in the headline numbers; it is examined separately by truncating user histories in a dedicated experiment [7].
- **Explicit feedback only, single test pass.** The system uses star ratings alone, ignoring the implicit signals (clicks, watch time) a production system would exploit;

and, by design, the test set was scored once, so test-retest variance is approximated by bootstrap confidence intervals rather than measured directly.

5.3 Future Work

The findings suggest clear next steps. First, since rating-optimal models proved weak at ranking, replacing the rating-trained SVD base with a ranking-trained one—Bayesian Personalised Ranking [13] or an explicit learning-to-rank head—should lift the hybrids on exactly the metric where they have the most headroom. Second, training LightGCN on the full user set with GPU acceleration would remove its coverage caveat and reveal the true ceiling of graph-based collaborative filtering on this data [12]. Third, incorporating implicit feedback would broaden the signal well beyond the sparse explicit ratings. A more ambitious direction, well aligned with the content of the course, is to cast the problem as link prediction over a richer user–item–genre–tag knowledge graph and to fuse structural embeddings with the content and collaborative signals already in place. Finally, on the engineering side, slimming the serialised collaborative models—which currently retain their full training sets—would let several heavy models be served simultaneously rather than one at a time.

In closing, this study provides concrete, reproducible evidence for the value of hybridisation in recommender systems: across rating accuracy, ranking quality, and a practical per-user case study, a learned combination of content-based and collaborative-filtering signals proved to be the most accurate and the most consistently robust approach of all those evaluated.

Bibliography

- [1] Francesco Ricci, Lior Rokach, Bracha Shapira, and Paul B. Kantor, editors. *Recommender Systems Handbook*. Springer, Boston, MA, 2011. doi: 10.1007/978-0-387-85820-3.
- [2] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web (WWW)*, pages 285–295, 2001. doi: 10.1145/371920.372071.
- [3] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009. doi: 10.1109/MC.2009.263.
- [4] Pasquale Lops, Marco de Gemmis, and Giovanni Semeraro. Content-based recommender systems: State of the art and trends. In *Recommender Systems Handbook*, pages 73–105. Springer, 2011. doi: 10.1007/978-0-387-85820-3_3.
- [5] Robin Burke. Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4):331–370, 2002. doi: 10.1023/A:1021240730564.
- [6] Jonathan L. Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1):5–53, 2004. doi: 10.1145/963770.963772.
- [7] Andrew I. Schein, Alexandrin Popescul, Lyle H. Ungar, and David M. Pennock. Methods and metrics for cold-start recommendations. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 253–260, 2002. doi: 10.1145/564376.564421.
- [8] Scott Deerwester, Susan T. Dumais, George W. Furnas, Thomas K. Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6):391–407, 1990.
- [9] Jesse Vig, Shilad Sen, and John Riedl. The tag genome: Encoding community knowledge to support novel interaction. *ACM Transactions on Interactive Intelligent Systems*, 2(3):1–44, 2012. doi: 10.1145/2362394.2362395.
- [10] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019. doi: 10.18653/v1/D19-1410.

- [11] Nicolas Hug. Surprise: A Python library for recommender systems. *Journal of Open Source Software*, 5(52):2174, 2020. doi: 10.21105/joss.02174.
- [12] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. LightGCN: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 639–648, 2020. doi: 10.1145/3397271.3401063.
- [13] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 452–461, 2009.
- [14] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992. doi: 10.1016/S0893-6080(05)80023-1.
- [15] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [16] F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015. doi: 10.1145/2827872.
- [17] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pages 173–182, 2017. doi: 10.1145/3038912.3052569.
- [18] Walid Krichene and Steffen Rendle. On sampled metrics for item recommendation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1748–1757, 2020. doi: 10.1145/3394486.3403226.